

AI+X Computing Acceleration: From Algorithms Development, Analysis, to Deployment: Project Report

Instructed by *Kaisheng Ma*

Student A, Student B

1 Introduction

Our project implements one new topology DragonFly, the corresponding routing algorithm and one flow-control technology Drain. Briefly, DragonFly is a topology where the hop count between nodes are low and thus achieves low latency. And our adaptive routing algorithm chooses from the minimal routing and the Valiant routing, which is a randomized algorithm that can balance loads in the network. However, both the minimal routing and the adaptive routing can cause deadlocks in the network when the traffic load is high, so we exploit Drain as our flow-control technology that can remove potential deadlocks in the network periodically. Our project includes the following highlights:

- We implemented the NoC technologies in two topics: topology & routing and flow-control. And they can combine together to give a good performance.
- We wrote over 1,000 loc in total. And the drain part is complex to implement, which took us much time coding and debugging.
- To combine the DragonFly and the Drain, we came up with our novel way to modify the DragonFly to make it appropriate for the Drain, and we named the new topology DrainFly.
- We in fact implement another topology 3D-torus along with its minimal and adaptive routing algorithm (see our source code). Although we abandon it in our project since its performance is much worse than DragonFly, we think it can be regarded as our extra work.

2 Architecture

2.1 Topology

Our topology, named DragonFly (Figure 1), is a hierarchical network with three levels: system, group and router. At group level, n routers in the same group are fully associated. At the system level, there are $2n + 1$ groups and every two groups are connected via one link. Specifically, the router with index i in the group j are connected to $j + 2i + 1$ and $j + 2i + 2$ via the global channels (module $2n + 1$). Based on the topology, we can compute some important metrics: Diameter = 3, Degree = $n + 1$, Avg Distance ≈ 3 , Cost = $O(n^3)$. We can see that DragonFly is a topology where nodes have higher radix in exchange of lower overall hop count. And the network is highly scalable since no matter how big the network size n is, the diameter remains 3. So DragonFly has two advantages: it exploit locality for near-neighbor traffic and has zero-load latency.

However, it also has a disadvantage in the low path diversity, since the number of shortest path between any two nodes is only 1.

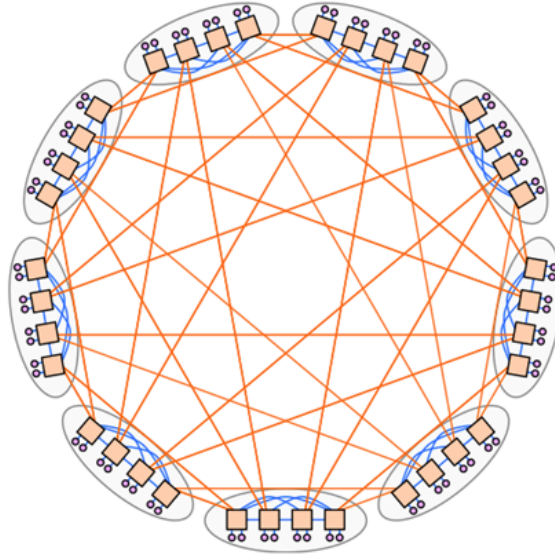


Figure 1: DragonFly

2.2 Routing

We implement three basic routing methods for the DragonFly topology: minimal, Valiant and adaptive. Based on the local link and global channel, the minimal routing can transfer the packet from the source node s attached to router R_s in group G_s to destination node d attached to router R_d in group G_d via at most two local and one global channels:

Step 1: If $G_s \neq G_d$ and R_s does not have a connection to G_d , route within G_s from R_s to R_a , a router that has a global channel to G_d .

Step 2: If $G_s \neq G_d$, traverse the global channel from R_a to reach router R_b in G_d .

Step 3: If $R_b \neq R_d$, route within G_d from R_b to R_d .

The minimal routing works well for load-balanced traffic, but results in very poor performance on adversarial traffic patterns where the sender are almost in group and the receiver are in another group. To balance the load in adversarial traffic pattern, Valiant's routing algorithm routes each packet to first a randomized intermediate group G_i and then to the final destination d . Valiant routing can transfer each packet with at most one additional local and one additional global channels compared to minimal routing:

Step 1: If $G_s \neq G_i$ and R_s does not have a connection to G_i , route within G_s from R_s to R_a , a router that has a global channel to G_i .

Step 2: If $G_s \neq G_i$ traverse the global channel from R_a to reach router R_x in G_i .

Step 3: If $G_i \neq G_d$ and R_x does not have a connection to G_d , route within G_i from R_x to R_y , a router that has a global channel to G_d .

Step 4: If $G_i \neq G_d$, traverse the global channel from R_y to router R_b in G_d .

Step 5: If $R_b \neq R_d$, route within G_d from R_b to R_d .

Valiant routing can balance the load in the network, but will cost more than the minimal routing. The adaptive routing combines the benefit between the minimal routing and adaptive routing by selecting between the two routing strategy based on the local virtual channel capacity. Denote q_{m-vc} , q_{nm-vc} as the capacity of virtual channel of the current link that the minimal routing and Valiant routing will tranverse, H_m , H_{nm} as the hop count of two routing algorithm, our adaptive routing goes simply as follows:

```

if ( $q_{m-vc}H_m \leq q_{nm-vc}H_{nm}$ )
    route minimally;
else
    route nonminimally;

```

2.3 Flow-control

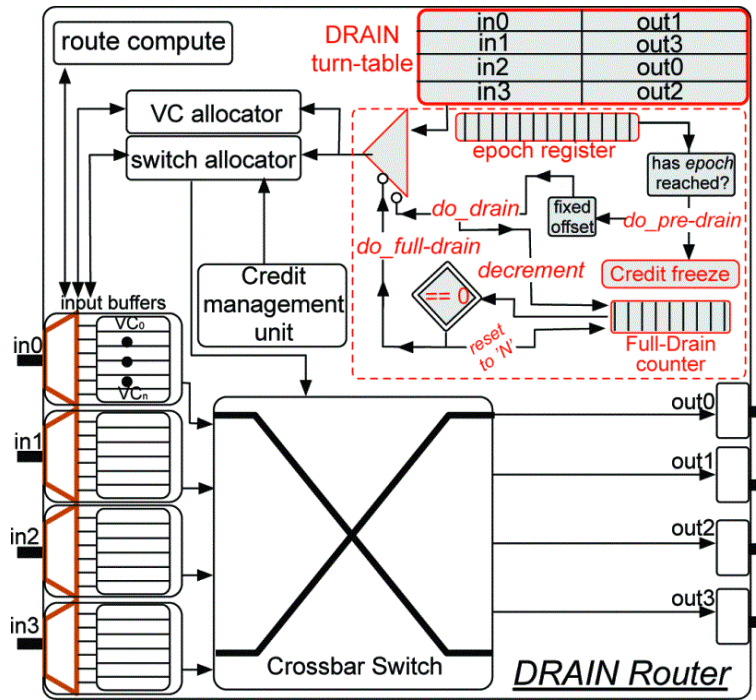


Figure 2: Drain

Considering that in the adaptive routing algorithm of DragonFly always incur deadlocks when the number of virtual channels per virtual network is low, we implemented DRAIN, a subtractive approach to remove potential deadlocks without needing to explicitly detect or avoid them. There are several important advantages of DRAIN as a method to solve deadlocks:

- Can be implemented regardless of the topology
- Routers do not need to communicate and coordinate their draining with each other
- Deadlock-free operation with low hardware complexity

It is worth to mention that **subtractive** here means that the network tries to remove the deadlocks at a given frequency, instead of detecting the deadlocks and making response. As in figure 2, the architecture of the routers is the normal architecture with drain-turn table that replace the switch allocator during the drain windows and an epoch register that decides in which stage the router is, the NORMAL stage, the pre-drain

window, or the drain window. Unfortunately, gem5 is an event-based simulator, so we cannot change the epoch register of a router without waking it up. As an alternative, we choose to force each router to refer to the number of Cycles of the whole network. The only requirement for the topology is that there must exist an Euler cycle (Figure 3), which passes each directional edge exactly once. We then construct the drain-turn table for each router basing on this Euler cycle. Unfortunately, the original DragonFly topology does not have any Euler cycle. So we modified this topology and come up with DrainFly, where we connect each $G_i R_{-1}$ to $G_{i+1} R_0$. Here $G_i R_{-1}$ means the last router in group G_i , while $G_{i+1} R_0$ denotes the first router in group G_{i+1} . In the pre-drain window, the routers freeze their credits and let the packets passing the links finish the current transmission. In the drain window, we only consider the escaping VC and force all the packets to move by the drain-turn window. This is guaranteed to be deadlock-free, since even without the normal stage, a packet can always reach its destination in that it transmit along an Euler cycle. However, it is essential that the duration of the NORMAL stage must be larger than the largest hops number to avoid livelocks in the network generated by DRAIN. In DrainFly, this number can be 6.

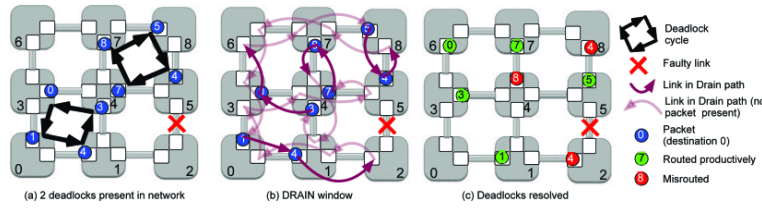


Figure 3: Euler circle in Drain

3 Implementation

3.1 Topology

Our implementation of DragonFly includes two parts: add links within groups and links among groups. We first divide routers into groups according to their index. Then we add links for every pair of routers in each group. Finally we add links among groups in the following way: for router (G_1, R_1) , we add two global links to $((G_1+1+2R_1) \bmod N, n-1-R_1)$ and $((G_1+2+2R_1) \bmod N, n-1-R_1)$ where $N = \text{\#number of groups}$, $n = \text{\#number of routers per group}$. However, we meet with a problem that the number of routers in garnet is limited to the power of 2, but by convention in DragonFly we should have $N = 2n + 1$ and the total number of routers $n(2n + 1)$. So to make our design fit for garnet, we pick $n = 4$ and cut off the final group and its links.

3.2 Routing

Our implementation of Routing is quite the translation of the algorithm in Sec. 2.2. For the minimal routing, we first check whether s and d are in the same group, if so, we output the direction outport of the local channel. Otherwise, we check whether s is linked to the group of d via a global channel. If so, we let the packet tranverses the global channel. Otherwise, we let the packet goes to the router in the same group as s that is linked to the group of d via a local channel. In this way, all packets can reach their destinations within 3 hops. For the Valiant routing, we first randomize a intermediate group for each routeInfo. Then we first try to transmit the packet to the intermediate group by the minimal routing. After the packet reaches the intermediate group, the packet then will be transmitted to the destination via the minimal routing. For the adaptive routing, we first retrieve the number of free virtual channels as q_{m_vc} and q_{nm_vc} on the way to the intermediate group and the destination for each group. And if $q_{m_vc}H_m \leq q_{nm_vc}H_{nm}$ we route the packet by minimal routing, otherwise by Valiant routing. And by experiments we found that pick $H_{nm} = 2H_m$ will yield slightly better results than $H_{nm} = 5, H_m = 3$.

3.3 Flow-control

As for the implementation, we add parameters `drain_turn_table_in_raw`, `drain_turn_table_out_raw`, `epoch`, `pre-drain-window`, `drain-window` to each router. Here variables `drain_turn_table_in_raw` and `drain_turn_table_out_raw` are initialized in the `makeTopology()` function of each topology object with the help of an algorithm of finding an Euler cycle in `configs/network/topologies/Eulerpath.py`. As mentioned above, the implementation of the DRAIN procedure will be quite tricky since gem5 is an event-based simulator and to ensure DRAIN is a simultaneous operation, we should not wait for each router to be waken up and conduct the DRAIN operation. So whenever a router is waken up, it will check the `curCycle` of the whole network and determine which stage the whole network is in and whether it should freeze its credits. If the `curCycle()` of the `get_net_ptr()` indicates that the whole system should start a DRAIN operation, the router will ask the network to call `do_drain()`, which is actually setting the network's stage to be `DRAIN_` and scheduling all the routers to wakeup during next `drain-window` cycles. Then if the router is waken up and finds the stage of the network to be `DRAIN_`, it then conduct the DRAIN operation. For all the routers, this will be simultaneous.

4 Experiments

4.1 Topology

We compare the performance of the DragonFly and the baseline topology Mesh_XY in the following experiment setting:

```
./build/NULL/gem5.opt configs/example/garnet_synth_traffic.py --network=garnet --num-cpus=32
--num-dirs=32 --topology={} --mesh-rows=8 --inj-vnet=0 --synthetic=uniform_random
--routing-algorithm={} --sim-cycles=10000 --injectionrate={} --vcs-per-vnet={}
```

where topology and routing-algorithm are chosen from Mesh_XY and DragonFly along with their corresponding minimal routing algorithms (1 and 6) with the injection rate ranging from 0.05 to 1.0. We conducted the experiment in two virtual channel settings: `vcs-per-net = 2` and 4. The results are shown in 4.

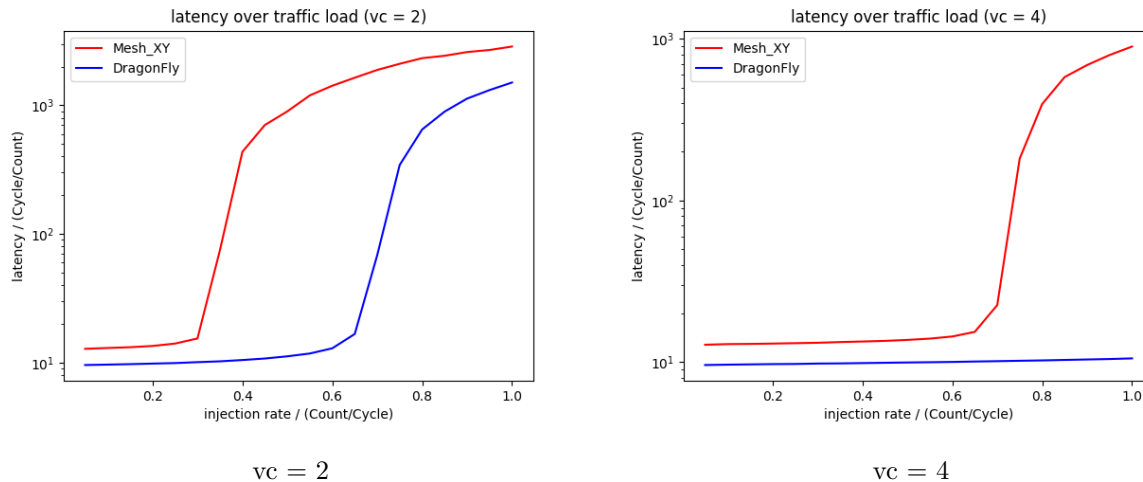


Figure 4

From the results, we can see that for both low and high traffic load, the DragonFly performs better than Mesh_XY in the network latency.

4.2 Routing

We compare the performance of the minimal routing as the baseline and the adaptive routing for DragonFly in the following experiment settings:

```
./build/NULL/gem5.opt configs/example/garnet_synth_traffic.py --network=garnet --num-cpus=32
--num-dirs=32 --topology=DragonFly --mesh-rows=8 --inj-vnet=0 --synthetic=uniform_random
--routing-algorithm={} --sim-cycles=10000 --injectionrate={} --vcs-per-vnet=2
```

```
./build/NULL/gem5.opt configs/example/garnet_synth_traffic.py --network=garnet --num-cpus=32
--num-dirs=32 --topology=DragonFly --mesh-rows=8 --inj-vnet=0 --synthetic=uniform_random --drain
--routing-algorithm={} --sim-cycles=10000 --injectionrate={} --vcs-per-vnet=2 --single-dest=15
```

where routing-algorithm is chosen from 6 and 7, which correspond to the minimal routing and adaptive routing with the injection rate ranging from 0.05 to 1.0. The former one is the uniform_random traffic pattern, while the latter one we assign the only destination router to make the traffic more congested, and regard it as the adversarial traffic pattern. Note that we use drain in the adversarial pattern where the network is easy to be totally congested (just like the deadlock). The results are shown in 5. From the results, we can see that for the uniform_random traffic, the adaptive routing performances almost equally to the minimal routing, while in the adversarial traffic pattern the adaptive routing performs better than the minimal routing in the latency.

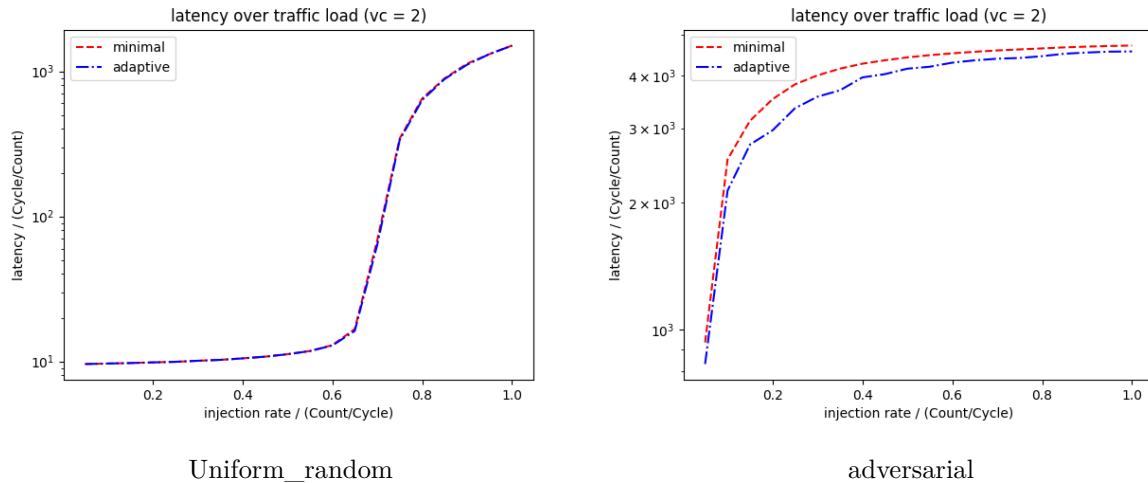


Figure 5

4.3 Flow-control

By experiments, we prove that DRAIN helps our adaptive routing algorithm of DragonFly to increase the reception rate and solve the deadlocks (baseline: without DRAIN). We experiment with sim-cycles=10000 and sim-cycles=100000. The experiments settings are in README.md and experiments/. The results are shown in 6.

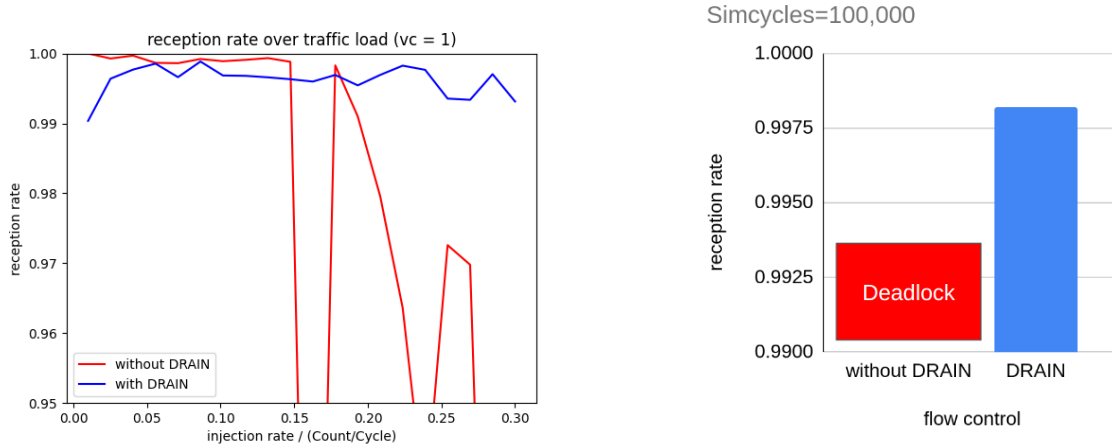


Figure 6

5 Analysis

5.1 Topology

When the traffic load is low, DragonFly has lower latency than the Mesh_XY because it has lower average hop count. When the traffic load is high, DragonFly outperforms Mesh_XY because its network has more links and thus more capacity. Besides, less hop count also makes the DragonFly harder to be congested and brings lower queuing latency than Mesh_XY.

5.2 Routing

When the traffic pattern is uniform_random, the occupation situation of virtual channels in all parts of the network is close, so the adaptive routing algorithm will choose to route minimally in most cases. Therefore, the minimal routing and the adaptive routing perform almost equally well. When the traffic pattern is adversarial, since every router has only one minimal path, the network will be very congested in the minimal routing. And our adaptive routing partly balance the load by routing the packet to intermediate node first. So the adaptive routing has less queuing latency and better performance than the minimal routing in the latency.

5.3 Flow-control

When the simulation cycles are low, the DRAIN helps to increase the reception rate slightly in that there are a few packets already get stuck. While when the simulation cycles are 100,000 and the number of the virtual channels per virtual network is low, the adaptive routing algorithm can get easily blocked by deadlocks after 50,000 cycles. Fortunately, DRAIN solves this issue and achieves a reception rate close to the one when the simulation cycles is low. This indicates that our DRAIN implementation greatly guarantee the correctness of our adaptive routing.

It is also remarkable that when the injection rate is extremely low, the reception rate is higher without DRAIN operation, this is natural since now the congestion seldom happens. Later as the injection rate increases, the congestion that leads to deadlocks has a more crucial impact on the reception rate of the packets.

6 Labor distribution

Student A (50%):

- Topology: DragonFly
- minimal & adaptive routing for DragonFly
- Experiment: Topology and Routing part

Student B (50%):

- Flow Control: Implementation of Drain
- Experiment: Drain part

7 Reference

DragonFly

- J. Kim, W. J. Dally, S. Scott and D. Abts, "Technology-Driven, Highly-Scalable Dragonfly Topology," 2008 International Symposium on Computer Architecture, Beijing, China, 2008, pp. 77-88, doi: 10.1109/ISCA.2008.19.

Drain

- Parasar, Mayank, Hossein Farrokhbakht, Natalie Enright Jerger, Paul V. Gratz, Tushar Krishna, and Joshua San Miguel. 2020. "DRAIN: Deadlock Removal for Arbitrary Irregular Networks." In 2020 IEEE International Symposium on High Performance Computer Architecture (HPCA), 447-60. San Diego, CA, USA: IEEE.